Abdelmalek Essaâdi University

National School of Applied Sciences of Al Hoceima

Data Engineering - DE 2

From Transformers to RAG: Building Smarter NLP Systems

Mini-Project Report

Transformers and Retrieval-Augmented Generation

Prepared by:

Salma Said
Mohamed Tamzirt

Supervised by:

Prof. Tarik Boudaa

Academic Year 2024–2025

Table of Contents

1 General Introduction

Natural Language Processing, commonly known as NLP, has become one of the key areas of artificial intelligence. Its main purpose is to enable machines to understand, interpret, and generate human language in a meaningful way. In recent years, this field has developed rapidly, moving from traditional rule-based systems to advanced deep learning architectures capable of processing large amounts of unstructured text.

One of the major breakthroughs in modern NLP was the introduction of the Transformer architecture. Before Transformers, recurrent models often struggled to capture long-range dependencies and were limited by slow sequential processing. Transformers helped address these limitations through self-attention mechanisms, which allow complete text sequences to be processed in parallel. This made it possible to train large pre-trained models that achieve strong performance across a wide range of NLP tasks.

However, despite their impressive ability to generate text, Large Language Models still have important limitations. They mainly rely on the knowledge they acquired during training, which means they cannot easily access private, recent, or domain-specific information. As a result, they may sometimes produce incorrect or hallucinated answers, especially when dealing with recent events or specialized topics. This project focuses on this issue by studying how external information can be integrated into the generation process.

The main objective of this report is to provide a practical exploration of Transformers and to demonstrate the implementation of a Retrieval-Augmented Generation system. The aim is to show how data engineering tools can be combined to build AI applications that are more reliable, more accurate, and better grounded in real knowledge.

This report is divided into two main parts. The first part introduces Transformers, Hugging Face, and common NLP tasks. The second part focuses on the RAG architecture and explains its implementation pipeline, starting from text chunking and vector search and ending with final answer generation. Finally, the report presents the experimental results and discusses the main observations.

2 Transformers and Hugging Face Environment

To understand modern NLP, it is useful to first look at the main ideas behind Transformers and the tools that make them easier to use in practice.

2.1 What are Transformers?

Transformers are neural network architectures introduced in 2017. They are based on the attention mechanism and avoid the sequential processing used by older Recurrent Neural Networks. This allows them to process tokens in parallel, which makes them more scalable and suitable for training on very large datasets.

2.2 The Attention Mechanism

The main innovation of the Transformer architecture is self-attention. When a model processes a sentence, self-attention helps it focus on the most relevant words around each token in order to understand its meaning in context. As a result, the model can build richer and more context-aware representations of language.

2.3 The Role of Pre-Trained Models

Training a Transformer model from scratch requires a very large amount of data and computational power. For this reason, many NLP applications rely on pre-trained models. These models are first trained on large text corpora so they can learn general language patterns. They can then be used directly for common NLP tasks or fine-tuned on smaller datasets for more specific needs.

2.4 Hugging Face Transformers

Hugging Face has become an important platform for the machine learning community. It provides the `transformers` library, which gives developers a simple and unified Python interface for using advanced models built with frameworks such as PyTorch and TensorFlow. With this library, powerful pre-trained models can be loaded and tested with only a few lines of code.

2.5 Tokenization Process

A neural network cannot work directly with raw text. It first needs the text to be converted into numerical input, and this step is handled by a tokenizer. The tokenization process generally follows these steps:

1. **Raw Text:** The original text provided by the user.
2. **Tokens:** The text is divided into words, sub-words, or characters, depending on the model vocabulary.
3. **Input IDs:** Each token is converted into a unique numerical ID.
4. **Model Input:** These IDs are transformed into tensors and then passed to the Transformer model.

When using a model from Hugging Face, it is important to load the tokenizer that belongs to the same model. This ensures that the text is prepared in the exact format expected by the model.

3 NLP Pipelines and Implemented Tasks

The first practical part of our project is based on Hugging Face pipelines. These pipelines make it easier to use NLP models because they manage the main steps automatically, including tokenization, model inference, and output formatting. This allowed us to focus more on understanding the results of each task rather than implementing every technical step manually.

3.1 Text Classification (Zero-Shot)

Objective: Classify a text into a category without training the model specifically on the proposed labels. **Model Used:** `facebook/bart-large-mnli` **Input:** A text describing a vector database, together with candidate labels such as "informatique", "sport", and "cuisine". **Output:** A probability score for each candidate label. **Interpretation:** The model correctly recognized that the text was mainly related to "informatique". This shows that zero-shot classification can understand the general meaning of a text and match it with suitable labels, even without task-specific training.

3.2 Sentiment Analysis

Objective: Identify whether a text expresses a positive, negative, or neutral sentiment. **Model Used:** `cardiffnlp/twitter-xlm-roberta-base-sentiment` **Input:** A sentence evaluating a project, such as *"Ce projet de NLP est très intéressant."* **Output:** A predicted sentiment label, such as "Positive", along with a confidence score, for example 0.95. **Interpretation:** This task shows how the model can capture the tone of a sentence and detect the emotion expressed in human language. In this example, the model identifies the sentence as positive because it contains an appreciative opinion about the project.

3.3 Question Answering

Objective: Extract the exact answer to a question from a given context paragraph. **Model Used:** `deepset/xlm-roberta-base-squad2` **Input:** A context paragraph describing the project goals, followed by a question asking what the project explores. **Output:** The exact answer extracted from the context, such as *"exploration des modèles Transformers."* **Interpretation:** This task illustrates extractive question answering. The model does not create a new answer from its own knowledge; instead, it searches inside the provided paragraph and returns the most relevant part of the text.

3.4 Automatic Summarization

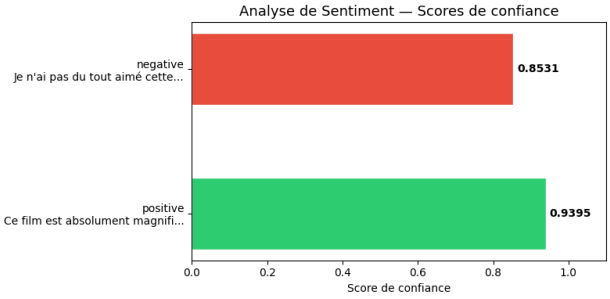
Objective: Transform a long text into a shorter version while keeping its main ideas. **Model Used:** `lincoln/mbart-mlsum-automatic-summarization` **Input:** A detailed paragraph about the history and impact of Transformers. **Output:** A short and coherent summary in one sentence. **Interpretation:** Summarization requires the model to understand the global meaning of the text and reformulate it in a shorter way. The goal is not only to reduce the length, but also to preserve the essential information in a clear and readable form.

4 Results and Analysis of Transformer Tasks

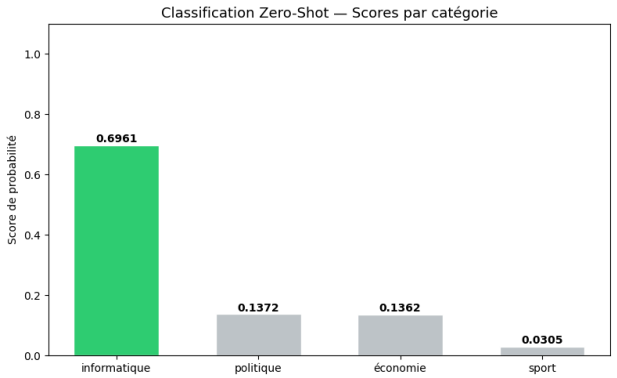
The implementation of Hugging Face pipelines produced consistent and accurate results across the selected tasks. By analyzing the outputs, we can clearly observe the strengths of pre-trained Transformer models.

4.1 Performance and Confidence Scores

In the sentiment analysis task, the model correctly classified clear input sentences with confidence scores above 0.90. In the zero-shot classification task, the model identified the "informatique" label with the highest probability. For the question answering task, the model successfully located the correct answer inside the text and returned its exact position. Similarly, the summarization pipeline produced a coherent summary that reduced the text length while keeping the main information.



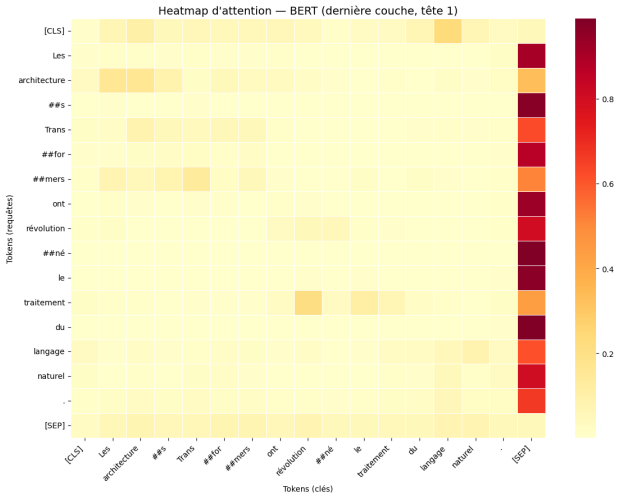
Sentiment Analysis Scores



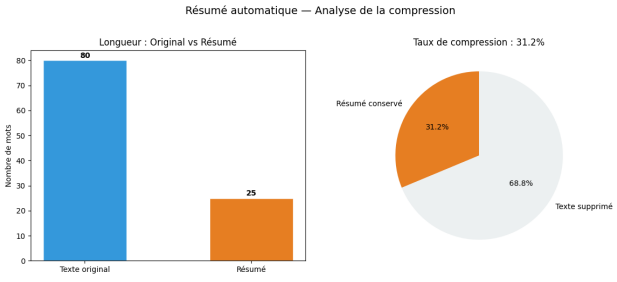
Zero-Shot Classification

4.2 Attention Visualization & Model Usefulness

To better understand how Transformer models process text, we visualized the attention weights of a BERT model. The visualization showed strong attention connections between subject nouns and their verbs, as well as between adjectives and the nouns they describe. This confirms that the model does not read words separately. Instead, it analyzes the relationships between words in the sentence.



BERT Attention Heatmap



Summarization Compression

The obtained results show that pre-trained Transformer models are highly flexible and useful for different NLP tasks. However, extractive models can only answer questions when the answer is explicitly present in the given text. This limitation naturally leads to the need for Retrieval-Augmented Generation systems.

5 Introduction to RAG and State of the Art

Although pre-trained language models are powerful, they still have important limitations when used in real-world environments. To address these limitations, many modern AI systems rely on Retrieval-Augmented Generation.

5.1 The Limitations of Standalone LLMs

Language models can be seen as snapshots of the knowledge available at the time they were trained. Because of this, they face three major challenges:

1. **Hallucinations:** When asked a question outside their knowledge base, an LLM may generate an answer that sounds confident but is actually incorrect.
2. **Outdated Knowledge:** A model trained in 2022 does not naturally know about events that happened in 2024.
3. **Lack of Domain-Specific Context:** Models do not automatically have access to private databases, internal project notes, or localized resources.

5.2 Retrieval-Augmented Generation Defined

RAG is an architectural pattern that improves the quality of a language model's response by grounding it in external sources. Instead of relying only on internal knowledge, a RAG system first retrieves relevant documents from a database and provides them to the model together with the user's question. The model then uses this context to generate a more accurate and natural answer.

5.3 Comparison of Approaches

To better understand the role of RAG, it is useful to compare it with other ways of adapting language models:

Table 1: Comparison of LLM Adaptation Techniques

Criteria	Fine-Tuning	Prompt Engineering	RAG
Mechanism	Modifies the model's internal weights using custom data.	Uses specific instructions to guide the model's output.	Retrieves external information and provides it as context.
Cost	High, because it usually requires significant computational resources.	Low, because it mainly depends on the quality of the prompt.	Medium, because it requires retrieval and indexing components.
Updates	Requires another training process.	Remains limited to the information already provided in the prompt.	Can be updated by modifying the external knowledge base.

5.4 Modern RAG Variants

In practice, RAG systems can be implemented in different ways:

- **Naive RAG:** A simple pipeline based on chunking, indexing, retrieval, and generation.
- **Advanced RAG:** An improved version that includes pre-retrieval techniques and post-retrieval re-ranking.
- **Modular RAG:** A more flexible architecture that uses separate modules for routing, searching, filtering, and iterative retrieval.

6 RAG Architecture and Pipeline

Designing a RAG system requires a structured data engineering approach. The pipeline connects unstructured text data to a vector search engine and then to a generative language model. This process can be divided into an offline ingestion phase and an online retrieval phase.

6.1 The Complete Pipeline

The architecture is composed of several main steps:

1. **Document Collection:** Gathering the raw textual knowledge base that the model will use to answer questions.
2. **Text Chunking:** Splitting long documents into smaller, overlapping segments called chunks. This step helps make retrieval more precise.
3. **Embedding Generation:** Processing each chunk with an embedding model in order to convert its semantic meaning into a dense numerical vector.
4. **Vector Indexing:** Storing these vectors in a specialized database optimized for fast similarity search.
5. **User Query Embedding:** When a user asks a question, the query is converted into a vector using the same embedding model.
6. **Top-k Retrieval:** The vector engine compares the query vector with the stored chunk vectors and returns the most similar chunks.
7. **Augmented Prompt Construction:** The retrieved chunks are combined with the user's question to create a structured prompt.
8. **Answer Generation:** The generative language model reads the augmented prompt and produces the final answer based on the retrieved context.

7 RAG Implementation

We translated the theoretical architecture into a working Python application using a Jupyter notebook environment. The implementation was designed to be simple, stable, and easy to understand.

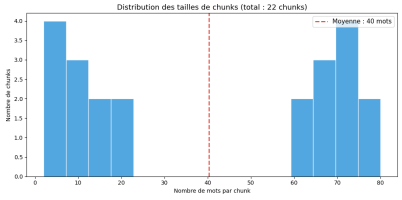
7.1 Tools and Corpus Strategy

The system was built using Python, `Sentence-Transformers`, `FAISS`, and `Transformers` via Hugging Face. We used a simplified and high-quality text corpus defined directly inside the code. This corpus contains different paragraphs about artificial intelligence, NLP, Transformers, and RAG architectures. Since the corpus was already organized into clear thematic passages, each paragraph naturally served as an individual chunk. This avoided the need for arbitrary character-based splitting.

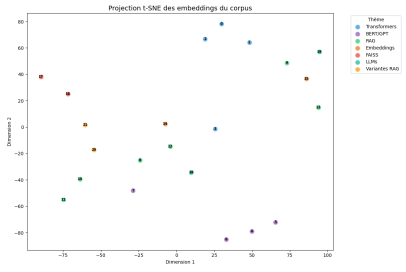
7.2 Embedding Generation & FAISS Indexing

To generate semantic vectors, we used the `paraphrase-multilingual-MiniLM-L12-v2` model. This model converts sentences into vectors with 384 dimensions. We encoded the whole corpus into a NumPy array so that sentences with similar meanings would have similar mathematical representations.

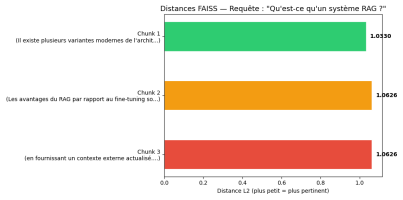
We then initialized a FAISS `IndexFlatL2` index, which uses Euclidean distance to perform exact nearest-neighbor search. The corpus embeddings were loaded into this index. During retrieval, the user's question is also converted into a 384-dimensional vector, and FAISS returns the distances and indices of the top-k closest chunks within milliseconds.



Chunks Size Distribution



t-SNE Embeddings Projection



FAISS Retrieval Distances

7.3 Building the Augmented Prompt

After retrieving the most relevant chunks, the system prepares them for the language model. We created a prompt template that gives clear instructions to the model. It includes the retrieved context and the user's question, while asking the model to answer only using the provided informa-

tion. This strict prompt structure helps reduce hallucinations. For answer generation, we used the `google/flan-t5-small` model.

8 Demonstration and Experimentation

To validate the implementation, we conducted experiments comparing the behavior of the generative model without RAG and with the RAG pipeline.

8.1 Example User Queries

We tested the system using domain-specific questions, such as:

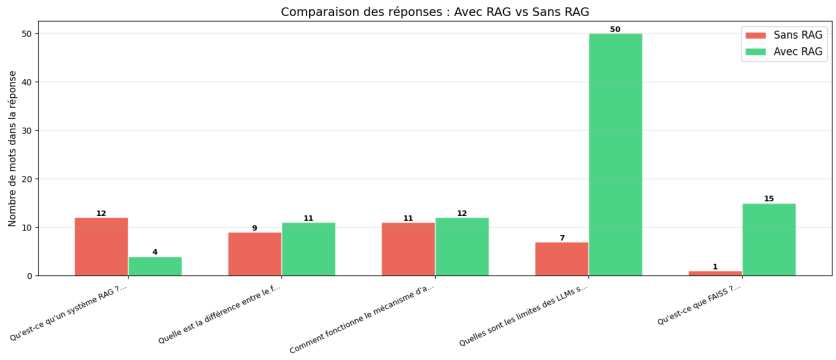
- *"Qu'est-ce qu'un système RAG ?"*
- *"Quelle est la différence entre le fine-tuning et le RAG ?"*

8.2 Comparison: Without RAG vs. With RAG

When the user asked about the difference between fine-tuning and RAG, the FAISS engine correctly understood the semantic intent of the question. It retrieved the two most relevant chunks from the corpus: one explaining that fine-tuning modifies the internal weights of a model at a high cost, and another explaining that RAG adds knowledge dynamically without retraining.

Execution Without RAG: When the question was passed directly to the `flan-t5-small` model without any additional context, the answer was short, generic, and lacked technical depth. The model relied only on its internal parameters and did not clearly mention concepts such as vector databases or dynamic knowledge updating.

Execution With RAG: When the retrieved context was added to the prompt, the model's answer became much more precise and grounded. It correctly explained that RAG uses an external corpus to retrieve information dynamically, while fine-tuning requires updating the model's neural network weights.



Comparison of Generated Responses: With RAG vs Without RAG

8.3 Interpretation of Results

The answer generated with RAG was more context-aware because the model was no longer forced to rely only on memorized knowledge. Instead, it behaved like an advanced reading comprehension system, using the retrieved text to build its response. This experiment shows that combining a lightweight generative model with an accurate retrieval engine can produce results that are more reliable and closer to those of larger models.

9 Discussion, Limitations, and Perspectives

Building and testing the RAG pipeline gave us a better understanding of both the strengths and the limitations of modern NLP systems.

9.1 What Worked Well

The integration between Sentence-Transformers and FAISS worked effectively. The vector search was able to retrieve relevant chunks even when the user's question was phrased differently from the source text. Also, moving from a standalone generation approach to a RAG-based approach helped reduce hallucinations in the tested examples. This shows that adding external context is a strong strategy for improving the reliability of generated answers.

9.2 System Limitations

Despite the success of the implementation, some limitations must be mentioned:

- **Small Corpus Size:** The system was tested on a small and carefully prepared dataset. In real enterprise environments, documents are usually larger, noisier, and more repetitive.
- **Simple Retrieval Strategy:** We used a basic Top-K semantic search. This method can sometimes retrieve chunks that are mathematically close to the query but do not contain the exact answer.
- **Generative Constraints:** The `flan-t5-small` model is computationally lightweight. This makes it easier to run, but it can limit the quality, fluency, and depth of long explanations, even when the retrieved context is relevant.
- **Lack of Quantitative Evaluation:** Our evaluation was mostly based on manual observation. We did not use advanced quantitative metrics to measure retrieval precision, recall, or answer faithfulness.

9.3 Improvements and Perspectives

To transform this mini-project into a more advanced and production-ready system, several improvements could be added:

- **Advanced Chunking:** Implement semantic chunking strategies that respect sentence boundaries and logical document structure.
- **Re-ranking Implementation:** Add a Cross-Encoder step after the initial FAISS retrieval to re-score the retrieved chunks and pass the most relevant text to the LLM.
- **Evaluation Frameworks:** Integrate frameworks such as RAGAS to evaluate context precision and answer faithfulness automatically.
- **Modular Architecture:** Explore Modular RAG designs, where the system can decide whether to search a vector database, call a web API, or query a SQL database depending on the user's intent.

10 General Conclusion

This mini-project allowed us to connect the theoretical foundations of Natural Language Processing with a practical data engineering implementation. Through the Hugging Face ecosystem, we explored how pre-trained Transformer models can be applied to several common NLP tasks, including sentiment analysis, zero-shot classification, question answering, and automatic summarization.

More importantly, this work helped us understand the limitations of standalone language models and how Retrieval-Augmented Generation can address some of them. By building a complete RAG pipeline, we were able to process textual knowledge, generate dense vector embeddings, store them in a FAISS index, and perform semantic retrieval in real time. The retrieved information was then used to enrich the prompt given to the generative model.

The comparison between answers generated with and without RAG showed the value of adding external context to the generation process. With RAG, the model produced responses that were more accurate, more reliable, and easier to trace back to the source information. Instead of relying only on static internal knowledge, the model was guided by retrieved content, which made its answers more grounded and relevant.

In conclusion, this project shows that RAG is a strong approach for building more dependable AI systems, especially in situations where the model needs access to specific or regularly updated knowledge. As data engineering and artificial intelligence continue to evolve, RAG-based architectures are likely to play an important role in enterprise AI applications. The skills developed in this project, particularly embeddings, vector search, semantic retrieval, and prompt augmentation, provide a solid foundation for building modern intelligent systems.